



Mapas

Herramientas de graficado y mapas

Es importante destacar que la representación gráfica de datos geoespaciales permite a las empresas y organizaciones obtener información clave para la toma de decisiones. Además, les brinda una visión más amplia de su entorno, tanto a **nivel local (microsistema) como global (macrosistema).**

Es momento de iniciar con la graficación de mapas. Para ello, GeoPandas proporciona una interfaz de alto nivel basada en la librería Matplotlib. Crear un mapa es tan sencillo como utilizar el método `plot()` de una GeoSeries o un GeoDataFrame.

■ Descargar archivos

Como primer ejemplo, vas a crear mapas utilizando información geoespacial de México. Para ello, trabajarás en Google Colab, como lo has hecho previamente, y utilizarás archivos de la **colección mexican-states**, disponibles en diferentes formatos. Estos archivos fueron creados para Los Angeles Times con datos de OpenStreetMap. Además, emplearemos el archivo **Tabulado_INEGI_2010.csv**, obtenido del sitio del INEGI.

Ve a la sección de archivos adjuntos de la plataforma y descarga los archivos. También podrás encontrar los enlaces en la sección de Material adicional.

Descomprime el archivo `mexican-states.zip` y sube todos los archivos de nombre `Mexican-states` a Google Colab.

■ Instalar e importar GeoPandas

Para trabajar con estos archivos, es fundamental tener instalada la biblioteca GeoPandas.

1. Si aún no la tienes, instálala ejecutando el siguiente comando en una celda:

```
!pip install geopandas
```

Esto asegurará que GeoPandas esté disponible en tu sesión y puedas aprovechar sus funcionalidades.

2. Importa GeoPandas junto con las otras bibliotecas que necesitarás para este ejemplo. Ingresas las siguientes líneas de código:

```
import geopandas
import matplotlib.pyplot as plt
import numpy as np
```

■ Procesar datos geoespaciales

Para ello, primero monta tu unidad de Google Drive y no olvides que debes cambiar el ***path*** para que coincida con el directorio en que guardaste los archivos.

El siguiente paso es leer los archivos que contienen la información geoespacial de los polígonos que delimitan los estados de México. Para ello te vamos a mostrar cómo leer archivos en dos formatos diferentes: shp y geojson.

Para ello, monta tu unidad de Google Drive y no olvides que debes cambiar el ***path*** para que coincida con el directorio en que guardaste los archivos.

1. Cargar los archivos shp.

```
from google.colab import drive
drive.mount('/content/gdrive')

import geopandas as gpd
mex = gpd.read_file('/content/gdrive/MyDrive/TLG-
Geovisualizacion/NOTEBOOKS/mexican-states.shp')
mex.columns
```

Estas últimas dos instrucciones cargan los datos del archivo *shp* en un **GeoDataFrame** llamado `mex`, que contiene la información geográfica de México y nos permiten examinar la estructura de sus columnas.

2. Cargar el archivo `estados_mx.geojson`.

```
from google.colab import drive
drive.mount('/content/gdrive')

import geopandas as gpd
mex = gpd.read_file('/content/gdrive/MyDrive/TLG-
Geovisualización/NOTEBOOKS/estados_mx.geojson')
mex.columns
```

Observa que el código que usamos es el mismo, GeoPandas se encarga de convertir los datos en el formato que vengan a un GeoDataFrame.

Ahora, el conjunto de datos incluye muchas columnas que no son relevantes para nuestro análisis. En este caso, sólo nos interesan el nombre del estado y su geometría. Para seleccionar únicamente estas columnas, usa:

```
mex[["name", "geometry"]]
```

	name	geometry
0	Aguascalientes	MULTIPOLYGON (((-102.7453 21.72005, -102.74859...
1	Colima	MULTIPOLYGON (((-103.48859 18.96489, -103.4884...
2	Tlaxcala	POLYGON ((-98.5415 19.46082, -98.54158 19.4607...
3	Ciudad de México	POLYGON ((-99.2787 19.1317, -99.28259 19.13821...
4	Morelos	POLYGON ((-98.90339 19.03848, -98.90061 19.038...
5	México	POLYGON ((-100.13794 20.04431, -100.13759 20.0...
6	Hidalgo	POLYGON ((-99.73527 20.56486, -99.73525 20.564...
7	Puebla	POLYGON ((-97.26671 19.38249, -97.26549 19.381...
8	Nuevo León	POLYGON ((-99.94766 23.49713, -99.94611 23.490...
9	Coahuila de Zaragoza	POLYGON ((-103.50895 25.27801, -103.50984 25.2...
10	Chihuahua	POLYGON ((-107.9995 26.44849, -107.96961 26.47...
11	Sonora	MULTIPOLYGON (((-114.67691 31.75636, -114.6759...

Figura 1. Columnas de interés seleccionadas.

Al revisar la columna **geometry**, observa que en la mayoría de los casos los estados están representados como **Polygon**, pero algunos otros aparecen como **MultiPolygon**.

■ Visualizar datos geoespaciales

1. Ahora que hemos cargado la información geoespacial, podemos graficarla utilizando la función `plot()` del **GeoDataFrame** de la siguiente manera:

```
mex.plot()
```



Figura 2. Mapa con los polígonos de los estados de la República Mexicana.

Esto genera un mapa con los polígonos de los estados de México.

2. Si lo único que quieres es graficar los contornos de los polígonos, puedes usar el siguiente código:

```
mex.boundary.plot()
```

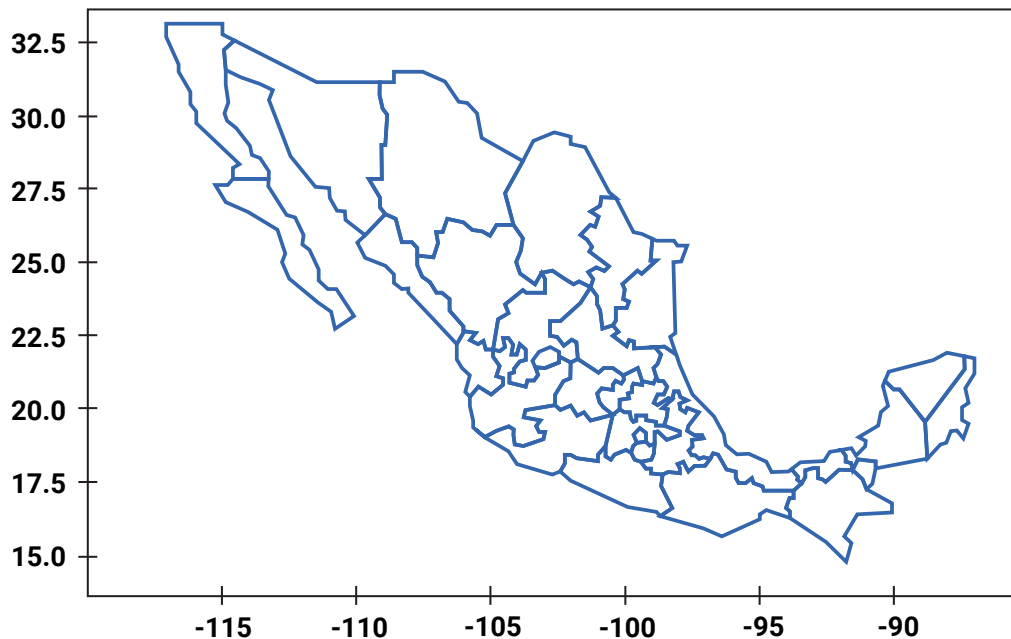


Figura 3. Mapa de contornos de los polígonos de los estados de la República Mexicana.

Mapas coropléticos

Se denomina mapa coroplético a aquellos que **representan datos cuantitativos de áreas geográficas mediante colores o patrones**. Estos mapas nos ayudan a comprender los datos asociados a regiones geográficas con mayor facilidad.

Hasta ahora, has visualizado un conjunto de polígonos de forma general, similar a lo que hiciste en la lección anterior. Sin embargo, ahora representarás cada estado con un color según el valor de otra variable para obtener un mapa coroplético de México.

1. Para ello, carga el archivo **Tabulado_INEGI_2010.csv**, que descargaste anteriormente y que contiene información sobre la población de cada estado en el año 2010. Luego, combina estos datos con el **GeoDataFrame** para integrar en un sólo conjunto la información geográfica y la población. Dado que los nombres de los estados coinciden en ambos **DataFrames**, puedes unir la información utilizando la función `merge()`.
2. El archivo de población contiene tres columnas: **hombres**, **mujeres** y **Total**. Para este ejemplo, toma en cuenta la columna **Total**. Verifica la estructura del nuevo **GeoDataFrame** mostrando sólo las columnas de interés.

```
Población = pd.read_csv('/content/gdrive/MyDrive/TLG-Geovisualizacion/NOTEBOOKS/Tabulado_INEGI_2010.csv')
mexgdf = mex.merge( población, left_on='name', right_on='Entidad')
mexgdf[['name', 'Entidad', 'geometry', 'Total']]
```

0	Aguascalientes	Aguascalientes	MULTIPOLYGON (((-102.7453 21.72005, -102.74859...	1184996
1	Colima	Colima	MULTIPOLYGON (((-103.48859 18.96489, -103.4884...	650555
2	Tlaxcala	Tlaxcala	POLYGON ((-98.5415 19.46082, -98.54158 19.4607...	1169936
3	Ciudad de México	Ciudad de México	POLYGON ((-99.2787 19.1317, -99.28259 19.13821...	8851080
4	Morelos	Morelos	POLYGON ((-98.90339 19.03848, -98.90061 19.038...	1777227
5	México	México	POLYGON ((-100.13794 20.04431, -100.13759 20.0...	15175862
6	Hidalgo	Hidalgo	POLYGON ((-99.73527 20.56486, -99.73525 20.564...	2665018
7	Puebla	Puebla	POLYGON ((-97.26671 19.38249, -97.26549 19.381...	5779829
8	Nuevo León	Nuevo León	POLYGON ((-99.94766 23.49713, -99.94611 23.490...	4653458
9	Coahuila de Zaragoza	Coahuila de Zaragoza	POLYGON ((-103.50895 25.27801, -103.50984 25.2...	2748391
10	Chihuahua	Chihuahua	POLYGON ((-107.9995 26.44849, -107.96961 26.47...	3406465
11	Sonora	Sonora	MULTIPOLYGON (((-114.67691 31.75636, -114.6759...	2662480

Figura 4. Columnas de interés seleccionadas para un mapa coroplético.

3. Ahora sí, grafica el mapa asignando colores a cada estado según la cantidad de habitantes registrados en la columna Total. Para ello, usa la función `plot()` indicando la columna que define la escala de colores.

```
mexgdf.plot(column='Total')
```

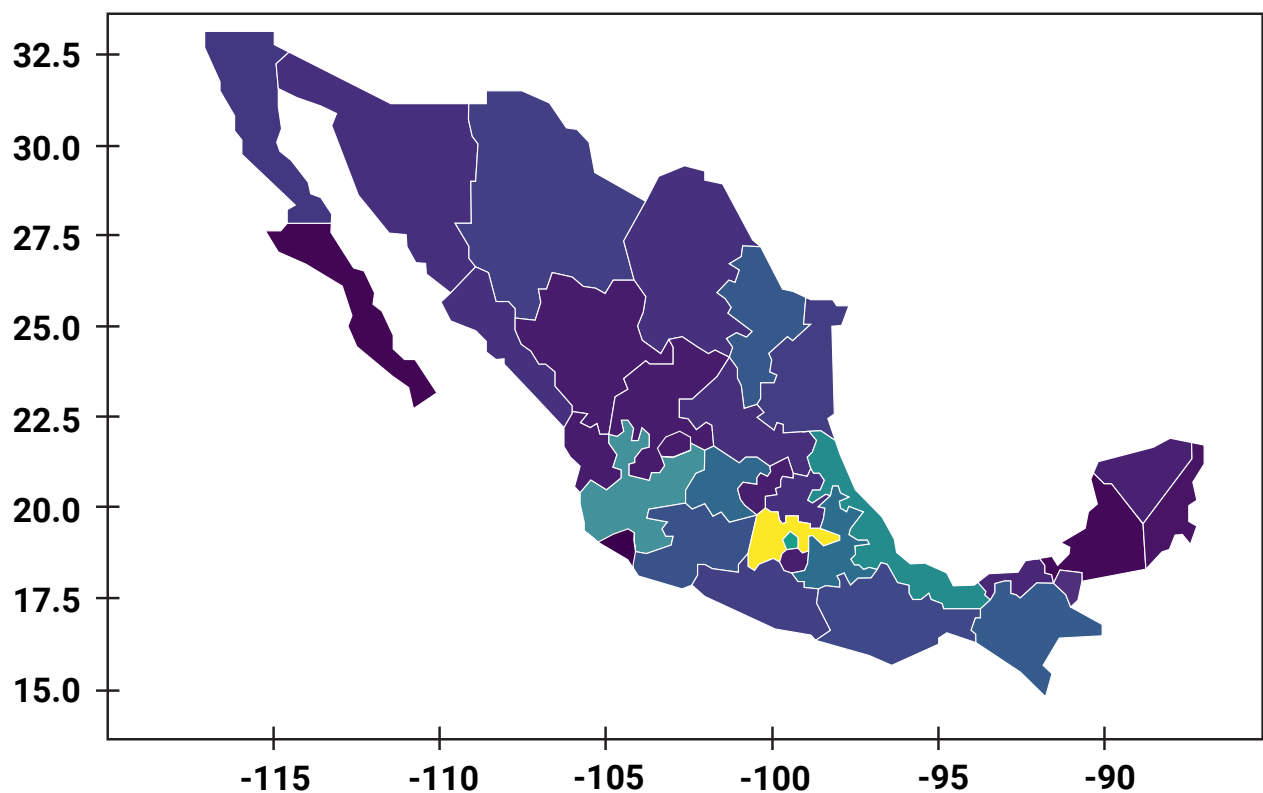


Figura 5. Colores asignados al mapa coroplético.

Personalizar el mapa

Leyenda

1. Para hacer más clara la interpretación del mapa, agrega una leyenda utilizando el parámetro **legend=True**. Asimismo, puedes personalizar la posición y el tamaño de la leyenda. Por ejemplo, colocándola a la derecha con un ancho del 8 %, como se muestra a continuación:

```
from mpl_toolkits.axes_grid1 import make_axes_locatable

fig, ax = plt.subplots(1, 1)
divider = make_axes_locatable(ax)
cax = divider.append_axes("right", size="8%", pad=0.1)

mexgdf.plot(column='Total', ax=ax, legend=True, cax=cax)
```

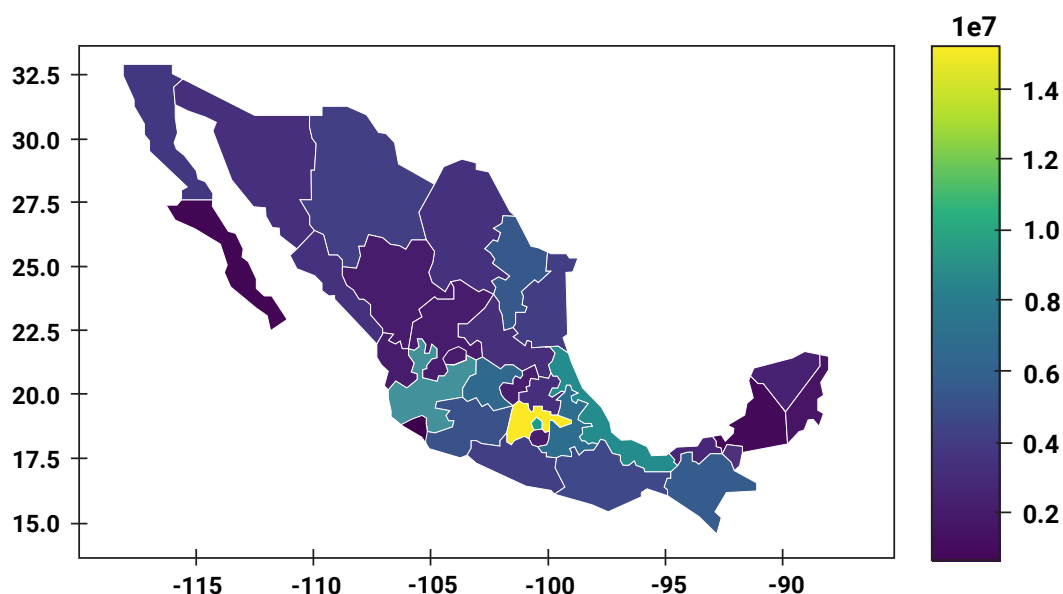


Figura 6. Leyenda de un mapa coroplético.

2. Prueba a moverla a la izquierda y cambiar el ancho a 15 %:

```
fig, ax = plt.subplots(1, 1)
divider = make_axes_locatable(ax)
cax = divider.append_axes("left", size="15%", pad=0.8)

mexgdf.plot(column='Total', ax=ax, legend=True, cax=cax)
```

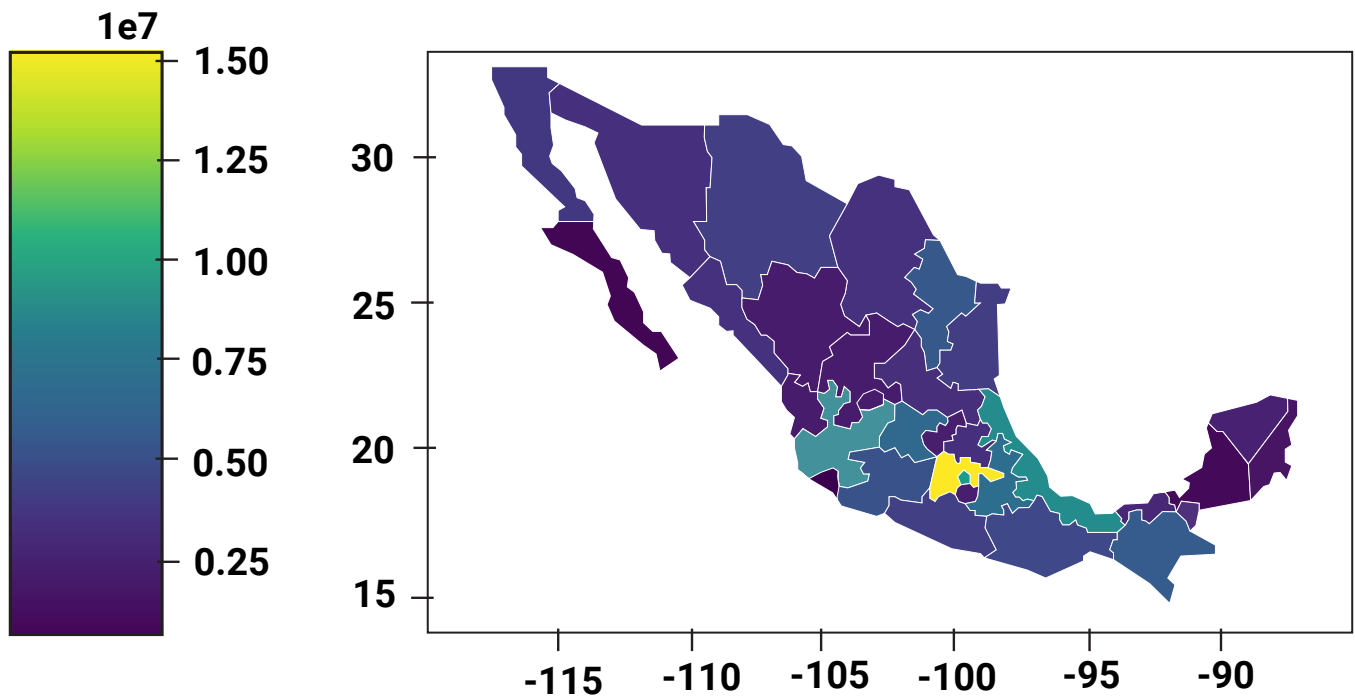


Figura 7. Leyenda de un mapa coroplético cambiado al lado izquierdo.

3. Ahora, agrega una etiqueta a la leyenda y colócala de forma horizontal:

```
fig, ax = plt.subplots(1, 1)
mexgdf.plot(column='Total', ax=ax, legend=True,
            legend_kwds={'label': "Población por estado",
                          'orientation': "horizontal"})
```

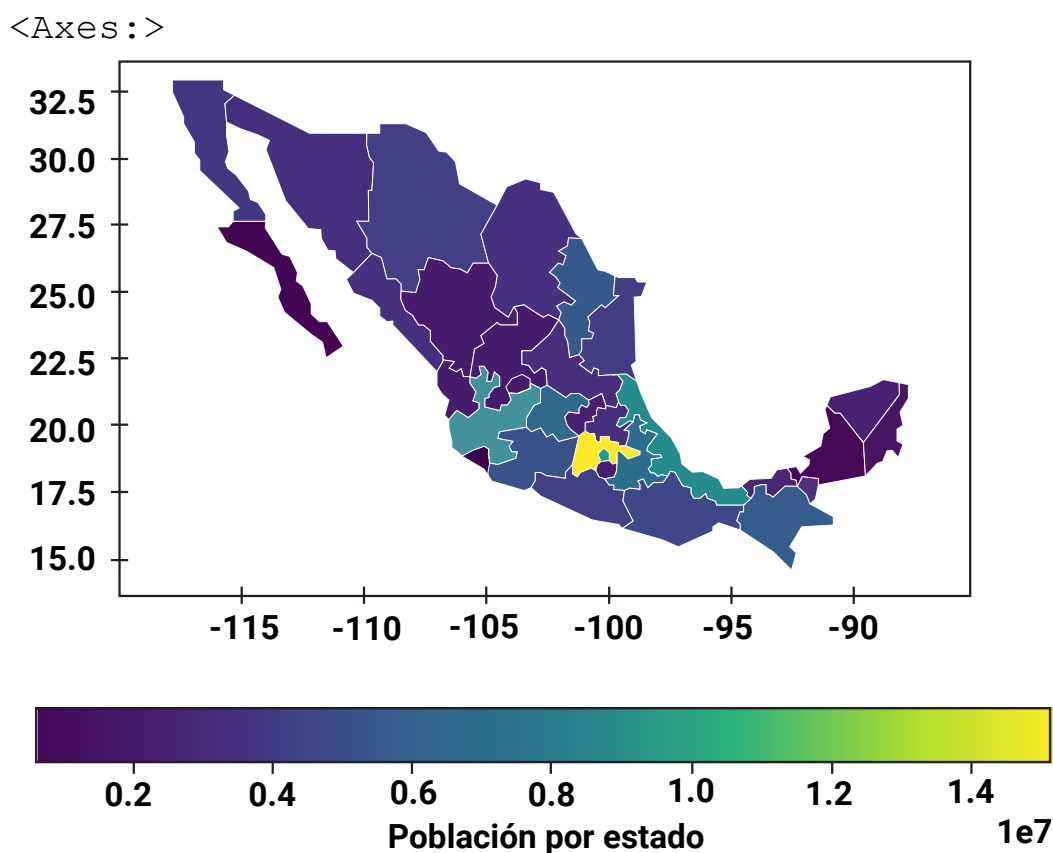


Figura 8. Mapa coroplético con etiqueta descriptiva.

Colores

Si deseas probar otra paleta de colores en el mapa, esto lo puedes lograr utilizando los **esquemas de color (*colormaps*)**.

Existen varias categorías de *colormaps*:

- **Secuenciales:** ideales para representar valores que van de menor a mayor.
- **Divergentes:** útiles cuando hay un punto medio de referencia.
- **Catagóricos:** para representar diferentes grupos o categorías.

Aquí tienes algunos ejemplos de mapas con diferentes *colormaps*:

```
mexgdf.plot(column='Total', cmap='tab20', legend=True)
mexgdf.plot(column='Total', cmap='OrRd', legend=True)
mexgdf.plot(column='Total', cmap='cool', legend=True)
```

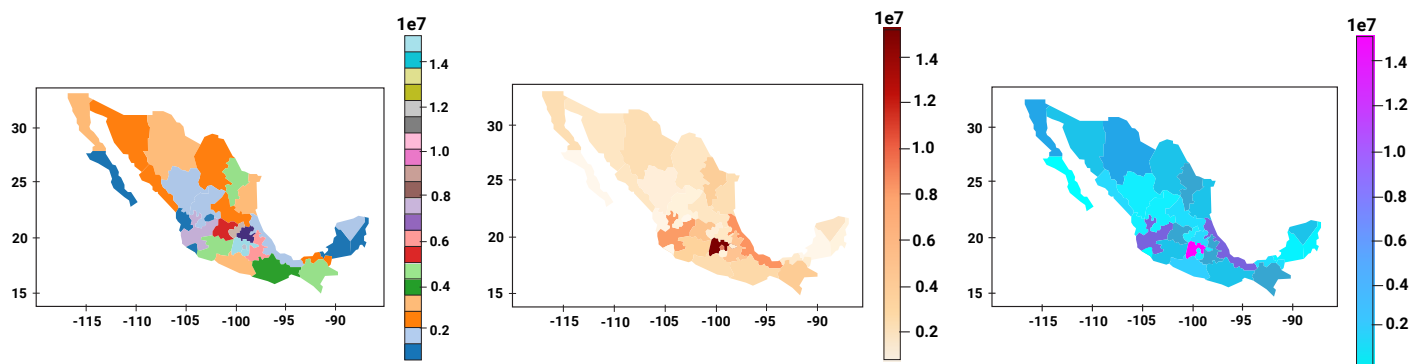


Figura 9. Uso de los esquemas de color (*colormaps*).

Si deseas aprender más sobre la elección de colores en mapas y otras opciones de visualización con **GeoPandas**, al final de este recurso puedes consultar la sección de Material adicional.



Ahora que has explorado herramientas para visualizar información geográfica en un GeoDataFrame, ya puedes representar distintos polígonos en un mapa. En nuestro ejemplo, mostramos a México y sus estados, pero este enfoque tiene muchas aplicaciones en situaciones del mundo real.

Por ejemplo, en empresas y organizaciones, los mapas pueden usarse para:

- Destacar áreas de alto riesgo en comparación con zonas seguras.
- Mapear la distribución de incidentes dentro de una planta de trabajo.
- Visualizar la cantidad de casos registrados de una enfermedad en diferentes regiones.
- Representar la densidad de población o el acceso a servicios en distintas localidades.

Las aplicaciones son prácticamente ilimitadas.

Antes de continuar, tómate un momento para reflexionar:

- ¿Qué otro tipo de información podrías representar de manera geográfica?
- ¿Cómo influiría la forma en que coloreas cada región en la interpretación de los datos?
- ¿Qué ventajas ofrece la visualización de datos en mapas en comparación a otras formas de representación?
- ¿Podrías utilizar mapas coropléticos para obtener una mejor comprensión de la información?

Capas

Hasta ahora, únicamente hemos graficado en una sola capa, por lo que a partir de este punto, aprenderemos a visualizar varias capas de información al mismo tiempo.

Para entender mejor cómo funcionan las capas, imaginemos que tienes varias hojas de acetato, cada una con un dibujo diferente que deja ver lo que hay debajo. Al superponerlas, los dibujos se combinan, permitiéndote verlos en conjunto. Esto es similar a lo que buscamos lograr con una superposición de capas en un mapa.

Podrías, por ejemplo, hacer un mapa donde se aprecien los estados de la República Mexicana y, sobre él, otro mapa con los ríos que atraviesan el país.

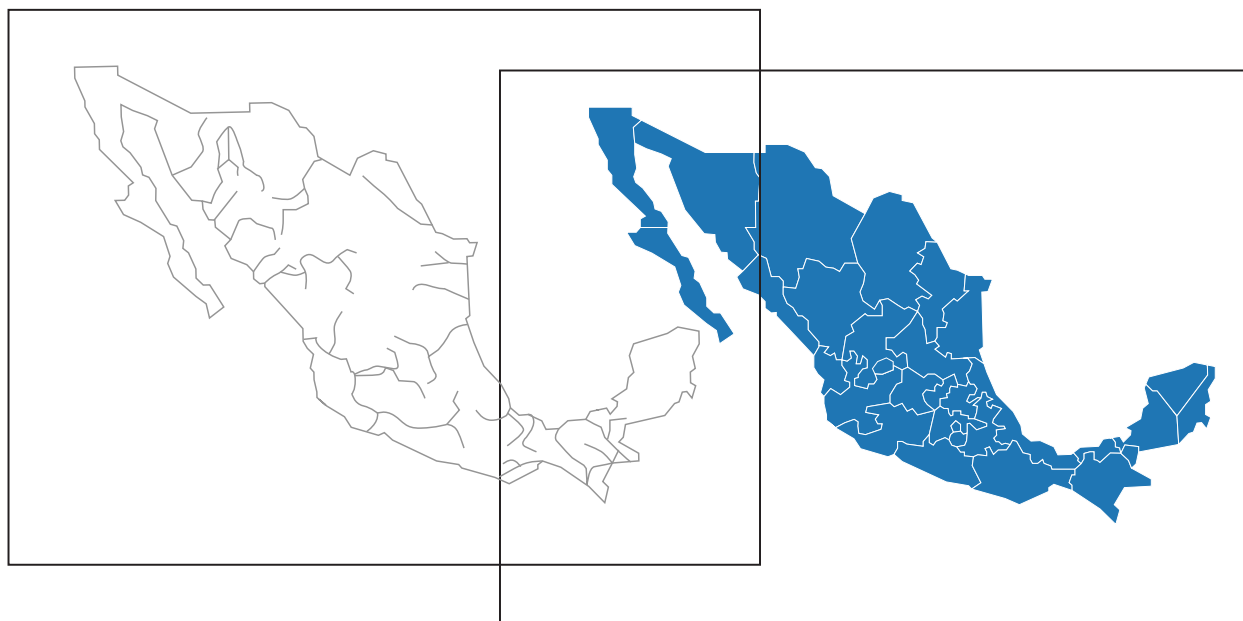


Figura 10. Visualización de mapas.

Sistema de referencia de coordenadas

Cuando se trabaja con mapas de varias capas, es fundamental asegurarnos de que todos tengan el mismo **SRC** o **Sistema de Referencia de Coordenadas**, también conocido como CRS (*Coordinate Reference System*, por sus siglas en inglés) antes de combinarlos.

Para comprender mejor este sistema, imaginemos cómo se representaría la superficie de la Tierra o una porción de ella en sólo dos dimensiones, ¿cómo podríamos plasmar una forma esférica en un plano?

Existen varias formas de hacerlo y, por lo general, se agrupan en **familias de proyecciones** que permiten conservar alguna característica importante, como las distancias, áreas o relaciones angulares.

El SRC nos ayuda a establecer la relación entre la proyección del mapa que estamos graficando con coordenadas y su ubicación en la Tierra. Por lo tanto, es importante **asegurarnos de que ambas gráficas compartan el mismo sistema**, ya que de otra forma nuestra visualización podría ser incorrecta.

En el ejemplo de los estados y los ríos, si usamos dos SRC diferentes, podríamos ver los ríos en un estado donde no están realmente o incluso fuera del mapa de México.

El atributo `crs` nos sirve también para **obtener información sobre el sistema de coordenadas**.

Podemos usar el comando `set_crs` para asignar un SRC o el comando `to_crs` para cambiar el SRC, como podemos ver en el siguiente ejemplo.

Si usamos el comando `to_crs` y nos aparece un error indicando que no había ningún CRS asignado, entonces es necesario usar el comando `set_crs`. En este caso, estamos asignando al `GeoDataFrame` `mexgdf` el mismo sistema de coordenadas que tiene el `GeoDataFrame` `mex`.

Observa el ejemplo:

```
mexgdf = mexgdf.to_crs(mex.crs)
mexgdf.crs
```

```
<Geographic 2D CRS: EPSG:4326>
Name: WGS 84
Axis Info [ellipsoidal]:
- Lat[north]: Geodetic latitude (degree)
- Lon[east]: Geodetic longitude (degree)
Area of Use:
- name: World.
- bounds: (-180.0, -90.0, 180.0, 90.0)
Datum: World Geodetic System 1984 ensemble
- Ellipsoid: WGS 84
- Prime Meridian: Greenwich
```

Figura 11. Ejemplo ejecutado del comando `to_crs`.

Graficando capas usando `plot` y `ax`

Como se revisó anteriormente, el comando `plot` se puede usar para graficar un `GeoDataFrame`. Ahora veamos cómo graficar otro `GeoDataFrame` sobre ese mismo mapa para aprender a agregar capas.

En temas anteriores, vimos cómo graficar un mapa de los estados de la República Mexicana, pero ahora queremos incluir más información en ese mismo mapa que indique la posición de las capitales de cada estado. Para ello, sigamos los pasos a continuación.

1. Lee el archivo `mexican-capitals.csv` que contiene información sobre los estados, las capitales, así como la latitud y longitud en la que se encuentran. En el siguiente ejemplo se está leyendo el archivo y convirtiendo la latitud y longitud en un objeto de tipo `Point` que después se utilizará como la geometría del `GeoDataFrame`:

```
df = pd.read_csv('mexican-capitals.csv')
capitales = geopandas.GeoDataFrame(df, geometry=geopandas.points_from_xy(df.Long, df.Lat))
```

```
capitales[:5]
```

	Estado	Capital	Lat	Long	geometry
0	Aguascalientes	Aguascalientes	21.880833	-102.296111	POINT (-102.29611 21.88083)
1	Baja California	Mexicali	32.663333	-115.467778	POINT (-115.46778 32.66333)
2	Baja California Sur	La Paz	24.142222	-110.310833	POINT (-110.31083 24.14222)
3	Campeche	San Francisco de Campeche	19.848611	-90.525278	POINT (-90.52528 19.84861)
4	Chiapas	Tuxtla Gutiérrez	16.753056	-93.115556	POINT (-93.11556 16.75306)

Figura 12. Ejemplo de lectura de archivo con información de latitud y longitud con `geopandas`.

2. El siguiente paso es asegurarnos siempre de que estamos usando el mismo SRC para ambos mapas. En el siguiente fragmento de código estamos usando el comando `set_src` para que nuestro `GeoDataFrame` de las capitales utilice el mismo sistema que el `GeoDataFrame` de los estados de la República.

3. A continuación, graficamos el mapa de los estados, pero guardamos la referencia en una variable de nombre mapa. Ahora, graficamos el segundo mapa con las capitales, pero usamos como valor para el atributo `ax` la variable `mapa`. Esto implica que el segundo GeoDataFrame se va a graficar dentro del mismo eje que graficamos el primero.

```
# Aseguramos que sea el mismo CRS
capitales = capitales.set_crs(mex.crs)
# Graficamos el mapa base
mapa = mex.plot()
# Graficamos los puntos correspondientes a las capitales
# del país
capitales.plot(ax=mapa, marker='v', color='red',
               markersize=20)
<matplotlib.axes._subplots.AxesSubplot at 0x7fd7d7989350>
```

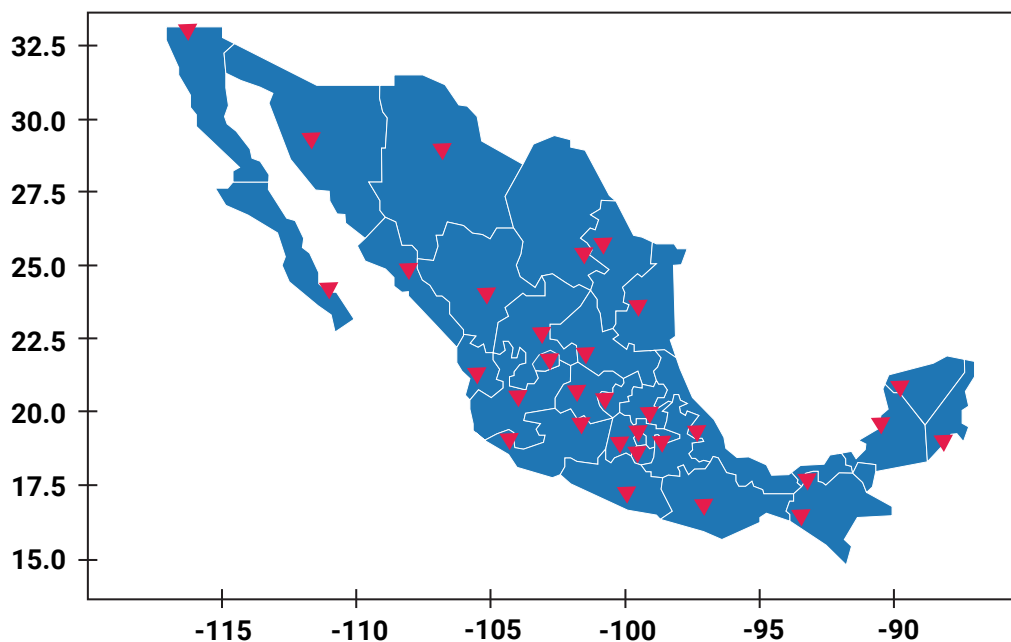


Figura 13. Mapa con capas usando GeoPandas.

Te compartimos el código completo en donde se localiza esta última parte:

```
import geopandas
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from google.colab import drive
drive.mount('/content/gdrive')

# actualizar la ruta

mex = geopandas.read_file('/content/gdrive/MyDrive/Colab
Notebooks/TLG-Geovisualización/NOTEBOOKS/mexican-states.
shp' )

df = pd.read_csv('/content/gdrive/MyDrive/Colab
Notebooks/TLG-Geovisualización/NOTEBOOKS/mexican-
capitals.csv' )
capitales = geopandas.GeoDataFrame(df,
geometry=geopandas.points_from_xy(df.Long, df.Lat))
capitales[:5]

# Aseguramos que sea el mismo CRS
capitales = capitales.set_crs(mex.crs)
# Graficamos ahora primero las capitales
mapa = capitales.plot(marker='v', color='red',
markersize=20)
# Y después el mapa completo
mex.plot(ax=mapa)

# Aseguramos que sea el mismo CRS
capitales = capitales.set_crs(mex.crs)
# Graficamos el mapa base
mapa = mex.plot()
```

```
# Graficamos los puntos correspondientes a las capitales
del país
capitales.plot(ax=mapa, marker='v', color='red',
markersize=20)
```

4. Observa que las capas en un nivel “superior” ocultan lo que hay debajo. Por tanto, si intentamos hacer esta misma gráfica, pero en el orden inverso, obtendremos un resultado diferente, como se muestra a continuación:

```
# Aseguramos que sea el mismo CRS
capitales = capitales.set_crs(mex.crs)
# Graficamos ahora primero las capitales
mapa = capitales.plot(marker='v', color='red',
markersize=20)
# Y después el mapa completo
mex.plot(ax=mapa)
```

<matplotlib.axes._subplots.AxesSubplot at 0x7fdfcaadca10>



Figura 14. Mapa con capas invertidas usando GeoPandas.

Graficando capas usando Matplotlib

Vamos a revisar otro método que también puede usarse para graficar capas sobre un mapa, pero usando la librería de visualización `matplotlib`.

Lo primero que tenemos que hacer es importar la librería para poder usar sus funcionalidades. Después, creamos un `subplot`, del cual recuperamos la figura y los ejes.

Una vez que los tenemos guardados en las variables `fig` y `ax`, realizamos las configuraciones necesarias para que el mapa se vea como queremos y usamos nuevamente el método `plot` del primer y segundo `GeoDataFrames`. No olvides indicar dentro de los parámetros del método `plot` que debe graficarse en los ejes del `subplot` que creamos al inicio. Finalmente, le pedimos a `pyplot` que muestre la gráfica compuesta.

Revisa con cuidado el siguiente código para identificar cada uno de los pasos que hemos descrito.

```
# Como vamos a usar objetos de matplotlib es necesario
importarlo
import matplotlib.pyplot as plt
# Creamos una sola figura y un subplot
fig, ax = plt.subplots()
# Le damos un título al mapa
ax.set_title('Capitales de la República Mexicana')
# El parámetro equal ajusta el tamaño de los ejes para que
tengan
# una correspondencia de 1 a 1
ax.set_aspect('equal')
# Graficamos el mapa usando los ejes que definimos arriba
mex.plot(ax=ax)
# Graficamos las capitales usando los mismos ejes
capitales.plot(ax=ax, marker='v', color='red', markersize=20)
# Mostramos el mapa
plt.show();
```

Capitales de la República Mexicana

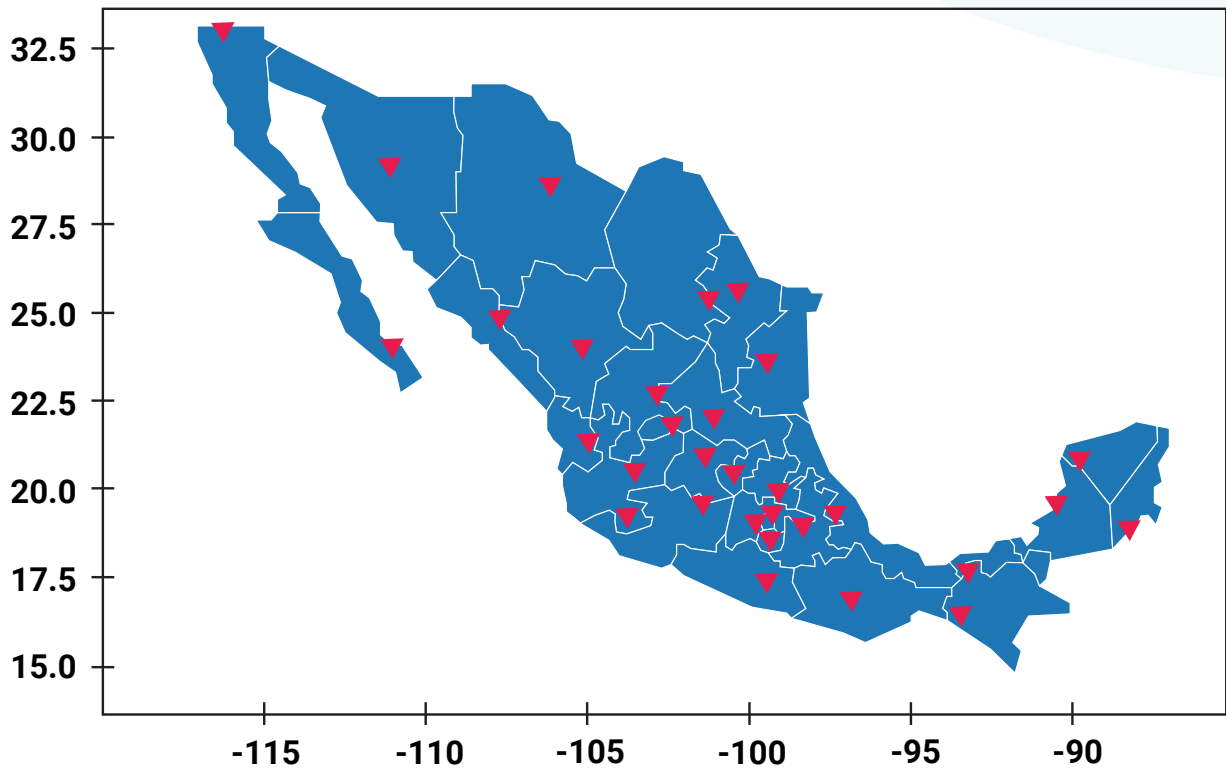


Figura 15. Mapa con capas en GeoPandas usando `matplotlib`.

Al igual que con el método anterior, el orden en que se grafican las capas es importante, por lo que primero se debe graficar el fondo y sobre él agregar los detalles.

En el siguiente ejemplo, puedes ver el resultado que se obtendría si se hubiera invertido el orden de las gráficas.

```
# Como vamos a usar objetos de matplotlib es necesario
importarlo
import matplotlib.pyplot as plt
# Creamos una sola figura y un subplot
fig, ax = plt.subplots()
# Le damos un título al mapa
ax.set_title('Capitales de la República Mexicana')
# El parámetro equal ajusta el tamaño de los ejes para que
tengan
# una correspondencia de 1 a 1
ax.set_aspect('equal')
# Graficamos primero las capitales
capitales.plot(ax=ax, marker='v', color='red', markersize=20)
# Graficamos el mapa sobre las capitales
mex.plot(ax=ax)
# Mostramos el mapa
plt.show();
```



Figura 16. Mapa con capas invertidas usando GeoPandas.

Controlar el orden de las capas

Es posible indicar el orden de las capas usando el atributo `zorder`, ten en cuenta que mientras más grande sea el valor, más arriba se encuentra la capa en la pila de capas.

Observa en el siguiente fragmento de código cómo, aunque las capitales se grafican primero, el valor de `zorder` hace que se pongan más “arriba” del mapa.

```
# Graficamos ahora las capitales en zorder 2
mapa = capitales.plot(marker='v', color='red', markersize=20,
                      zorder=2)
# Y después el mapa completo en zorder 1
mex.plot(ax=mapa, zorder=1)
```

<matplotlib.axes._subplots.AxesSubplot at 0x7fdfca8aecd0>

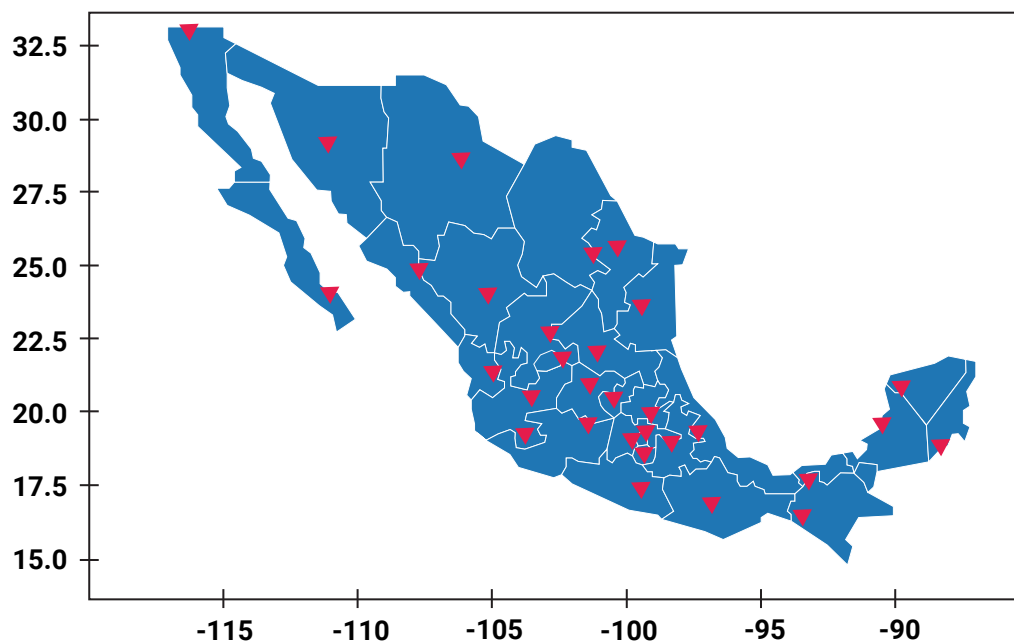


Figura 17. Mapa con capas invertidas usando `zorder`.

Se pueden agregar tantas capas como se requieran, siempre tomando en cuenta que el orden es importante para poder apreciarlas correctamente.

Ahora imaginemos que queremos el mapa de la República Mexicana como fondo, después las fronteras de los estados y, finalmente, la posición de las capitales. ¿Cuál debería ser el orden de las capas?, ¿cuál sería el orden usando `zorder`?

A continuación, te compartimos el código para realizar lo antes mencionado:

```
# Graficamos ahora las capitales en zorder 3 para que esté hasta arriba
mapa = capitales.plot(marker='v', color='red',
                      markersize=20, zorder=3)
# Y después el mapa completo en zorder 1 para que sea el fondo
mex.plot(ax=mapa, zorder=1)
# Finalmente, graficamos los bordes en zorder 2 para que queden en medio
mex.boundary.plot(ax=mapa, zorder=2, color='white')
```

<matplotlib.axes._subplots.AxesSubplot at 0x7fdfca12f490>

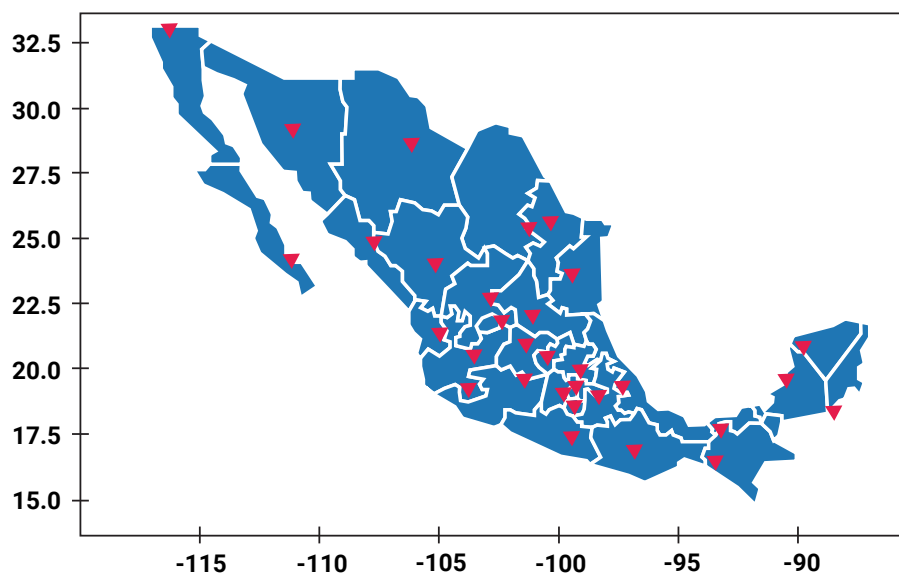


Figura 18. Mapa con varias capas usando `zorder`.



Ya revisamos que los mapas pueden contener una o varias capas y que el orden en el que se grafican es importante para lograr visualizar la información que se requiere.

Ahora tómame un momento para reflexionar:

- ¿Cuál consideras la mejor forma de graficar capas?
- ¿Consideras que es más fácil graficar las capas en orden o usar el atributo zorder en ciertas situaciones?
- ¿Ya pensaste en algún tipo de información geoespacial que podrías graficar usando capas?

Material adicional

Archivos de descarga para el ejercicio:

- Mexican-states <https://github.com/jschleuss/mexican-states>
Para obtener los archivos, haz clic en el botón **Code** y selecciona **Download ZIP**.
- Archivo simplificado de los estados de la República Mexicana: https://drive.google.com/file/d/1hP5e3dcbVB-_sfBqQS1q_RS9TmgrwLsR/view
- Tabulado_INEGI_2010.csv: <https://www.inegi.org.mx/app/tabulados/default.html?nc=mdemo02>

Matplotlib. (s.f.) *Choosing Colormaps in Matplotlib*. [Elegir mapas de colores en Matplotlib]. Matplotlib. Recuperado el 21 de marzo del 2025 de: <https://matplotlib.org/stable/users/explain/colors/colormaps.html>

Nota: Los enlaces a las páginas web incluidos en este Material adicional fueron consultados y verificados por última vez el 21 de marzo del 2025 y su disponibilidad depende del autor.

Glosario

- **Colormap:** Esquema de colores utilizado para visualizar datos en mapas y gráficos.
- **GeoDataFrame:** Estructura de datos de GeoPandas que permite manejar información geoespacial, similar a un DataFrame de pandas pero con una columna de geometría.
- **Geometría (Geometry):** Información espacial que define la forma de los objetos en un GeoDataFrame, como polígonos o multipolígonos.
- **GeoPandas:** Biblioteca de Python que extiende las funcionalidades de pandas para trabajar con datos geoespaciales de manera sencilla y eficiente.
- **Google Colab:** Plataforma en línea que permite ejecutar código en Python en un entorno basado en Jupyter Notebook.
- **Leyenda (Legend):** Elemento del mapa que explica los colores o símbolos utilizados, facilitando su interpretación.
- **Mapas coropléticos:** Mapas que muestran datos cuantitativos mediante variaciones de color para representar valores en distintas áreas geográficas.
- **Merge:** Función de pandas que permite combinar dos DataFrames basándose en una columna en común.
- **MultiPolygon:** Conjunto de varios polígonos que representan una sola entidad geográfica.
- **Polígono (Polygon):** Representación geométrica de una región con límites bien definidos en un mapa.

Bibliografía

Datavizcatalogue. (s.f). *Mapa Coroplético*. Datavizcatalogue. Recuperado el 21 de marzo del 2025 de: https://datavizcatalogue.com/ES/metodos/mapa_coropletico.html

Geeksforgeeks. (01 de agosto del 2024). *Matplotlib.pyplot.legend() in Python*. Geeksforgeeks. Recuperado el 21 de marzo del 2025 de: <https://www.geeksforgeeks.org/matplotlib-pyplot-legend-in-python/>

GeoPandas <https://geopandas.org>

GeoPandas (s.f). *GeoDataFrame*. GeoPandas. Recuperado el 21 de marzo del 2025 de: <https://geopandas.org/en/stable/docs/reference/geodataframe.html>

GeoPandas (s.f). *Mapping and plotting tools*. [Herramientas de mapeo y gráficos] GeoPandas. Recuperado el 21 de marzo del 2025 de: https://geopandas.org/en/stable/docs/user_guide/mapping.html

Google Colab <https://colab.research.google.com>

Matplotlib. (s.f.) *Colormap reference*. [Referencia de mapa de colores]. Matplotlib. Recuperado el 21 de marzo del 2025 de: https://matplotlib.org/stable/gallery/color/colormap_reference.html

Pandas. <https://pandas.pydata.org/>

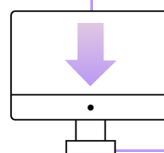
Pandas. (s.f). *pandas.merge*. Pandas. Recuperado el 21 de marzo del 2025 de: <https://pandas.pydata.org/docs/reference/api/pandas.merge.html>

Shapely. (s.f). *Geometry*. Shapely 2.0.6 documentation. Recuperado el 21 de marzo del 2025 de: <https://shapely.readthedocs.io/en/stable/geometry.html>

Shapely. (s.f). *shapely.MultiPolygon*. Shapely 2.0.6 documentation. Recuperado el 21 de marzo del 2025 de: <https://shapely.readthedocs.io/en/2.0.6/reference/shapely.MultiPolygon.html>

Shapely. (s.f). *shapely.Polygon*. Shapely 2.0.6 documentation. Recuperado el 21 de marzo del 2025 de: <https://shapely.readthedocs.io/en/2.0.6/reference/shapely.Polygon.html>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 21 de marzo del 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.